

**UNIVERSIDAD PERUANA DE CIENCIAS
APLICADAS****FACULTAD DE INGENIERÍA****Construcción y visualización de Curvas de Bézier**

Curso : Álgebra Lineal (1AMA0331)
NRC : 4753
Docente : Nedin Fernandez Quispe
Grupo N° : 4
Integrantes : Arapa Titi Victor Piero (u202412310)
Condori Apaza Cristian Over (u20241e626)
Manrique Anaya Rai Jeferson (u20241e410)
Neyra Salazar Alvaro Alberto (u202211741)

2026-10

1. Índice

1. Índice	2
2. Plan de Actividades	3
2.1. Reunión 1	3
2.2. Reunión 2	3
3. Introducción	4
4. Objetivo General	6
4.1. Objetivos Específicos	6
5. Fundamento Teórico	7
5.1. Ejemplo aplicativo 1	16
5.2. Ejemplo aplicativo 2	17
6. Código del programa (Avance del 30%)	18
7. Pantallazos de la ejecución del programa y su explicación	21
7.1. Imagen 1 y su explicación	21
7.2. Imagen 1 y su explicación	21
7.3. Imagen 2 y su explicación	22
7.4. Imagen 3 y su explicación	23
7.5. Imagen 4 y su explicación	23
7.6. Imagen 5 y su explicación	23
.....	23
• Gráfico resultante después de la compilación del código antes descrito.	23
8. Bibliografía	23
9. Anexos	25

2. Plan de Actividades

2.1. Reunión 1

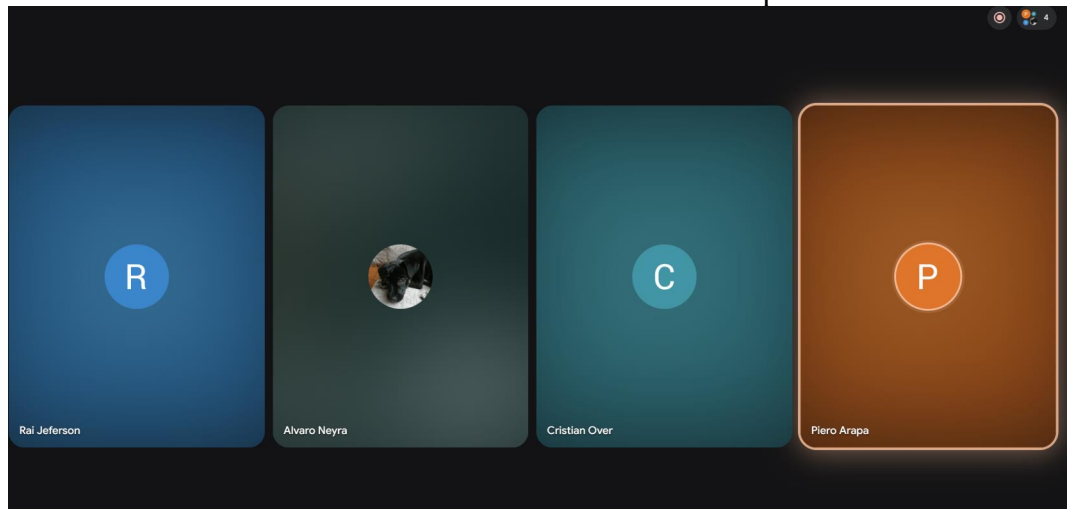
- Domingo 12 de abril:
 - **Investigación Teórica:** Estudio de los Polinomios de Bernstein como base del espacio vectorial y su diferencia con el algoritmo de De Casteljaú.
 - **Definición de Herramientas:** Configuración del entorno de desarrollo (PyCharm, Python 3.x) e instalación de dependencias críticas (numpy, matplotlib).

The image is a composite of two screenshots. The top screenshot shows a Google Meet interface. The main window displays a presentation slide titled "Estás presentando" (You are presenting) with instructions on how to share the screen. The sidebar on the right shows a list of participants: Alvaro Neyra, Cristian, and Piero Arapa. The bottom screenshot shows a YouTube video titled "Bezier curve example in computer graphics" by the channel "MADE_EASY_FOR_YOU". The video content shows a hand-drawn derivation of the Bezier curve equation. The text on the screen includes:

Design a Bezier curve controlled by 4 points
 $A(1, 1)$, $B(2, 3)$, $C(4, 3)$ and $D(6, 4)$.
 $P_0 = (0, 1)$ control points = 4
 $P_1 = (3, 3)$ degree = No. of CP - 1 = 4 - 1 = 3
 $P_2 = (4, 3)$ $\therefore d = 3 \rightarrow$ cubic curve
 $P_3 = (6, 4)$
 $P(\mu) = P_0(1-\mu)^3 + P_1 3\mu(1-\mu)^2 + P_2 3\mu^2(1-\mu) + P_3 \mu^3$
 $x(\mu) = 1(1-\mu)^3 + 2 \times 3\mu(1-\mu)^2 + 4 \times 3\mu^2(1-\mu) + 6\mu^3$
 $= (1-\mu)^3 + 6\mu(1-\mu)^2 + 12\mu^2(1-\mu) + 6\mu^3$

2.2. Reunión 2

- Domingo 19 de abril:
 - **Desarrollo de la Interfaz:** Implementación de la entrada dinámica de puntos y el renderizado estático de la curva.
 - **Programación de la Animación:** Configuración de FuncAnimation para visualizar la "influencia" de los vectores de control en tiempo real.



3. Introducción

Las curvas de Bézier surgieron hacia la década de los años 1960 como una solución a una pregunta fundamental en el diseño y el modelado geométrico: ¿Cómo describir matemáticamente una curva suave, controlable y estable? Dado que las herramientas geométricas clásicas, como las rectas o las circunferencias, resultaban insuficientes para modelar formas complejas, este sistema fue ideado por Pierre Bézier para facilitar el trazado de dibujos técnicos en el diseño aeronáutico y de automóviles. Desde entonces, este método de descripción matemática se comenzó a utilizar con gran éxito en los primeros programas de diseño asistido por computadora (Profesores UPC, 2022).

Históricamente, el desarrollo de estas curvas fue llevado a cabo de forma independiente por dos matemáticos que trabajaban para empresas automotrices diferentes: Pierre Bezier y Paul Casteljaou, quienes llegaron exactamente al mismo resultado geométrico, pero a través de caminos distintos.

Por un lado, Paul De Casteljaou abordó el problema desde una perspectiva puramente geométrica, lo que dio origen al algoritmo de De Casteljaou. Este es un método recursivo, visual y numéricamente muy estable para construir y evaluar estas curvas mediante la interpolación lineal de segmentos entre los puntos de control a una distancia proporcional a un parámetro t (regCode, 2021).

Por su parte, Pierre Bézier solucionó el problema desde una perspectiva más analítica, apoyándose fuertemente en el álgebra lineal. En la formulación de Bézier, la curva se define matemáticamente como una combinación lineal de los puntos de control espaciales, donde los coeficientes que ponderan la influencia de cada punto son los polinomios de Bernstein (Prautzsch et al., 2005). El uso de los polinomios de Bernstein es la verdadera clave del modelo, ya que otorgan a la curva sus propiedades analíticas y geométricas más valiosas, como el control global de la forma, la infinita derivabilidad (suavidad) y la garantía de estabilidad de que la curva no oscila, manteniéndose siempre estrictamente dentro de la envolvente convexa formada por el polígono de control (Profesores UPC, 2022).

Hoy en día, las curvas de Bézier han trascendido ampliamente el ámbito automotriz del que surgieron para convertirse en una herramienta matemática esencial que cualquier arquitecto o profesional del diseño conoce y aplica en su trabajo diario. Su inmenso impacto en la tecnología moderna se evidencia en su uso central en todo tipo de programas de diseño (como AutoCAD, Corel y Adobe Illustrator) y en lenguajes de impresión como PostScript (Galdames, 2011), además de ser el núcleo de disciplinas como la tipografía digital, el modelado industrial y las animaciones 3D (Profesores UPC, 2022). Dado que resulta altamente razonable y motivador el aprendizaje de las matemáticas necesarias para el desarrollo de estas curvas (Galdames, 2011), en el presente proyecto se construirá un programa interactivo que materialice estos conceptos. El software permitirá al usuario ingresar puntos de control libremente para que el sistema calcule la expresión paramétrica basada en los polinomios de Bernstein, con el fin de evaluar numéricamente y representar gráficamente la curva, demostrando así de forma visual cómo la variación de los puntos altera su morfología.

4. Objetivo General

Desarrollar un programa en python informático interactivo que permita la generación, el cálculo matemático y la visualización gráfica de curvas de Bézier a partir de un conjunto de puntos de control ingresados por el usuario, mediante la aplicación de los polinomios de Bernstein como base algebraica. Este software servirá como herramienta didáctica y de diseño para comprender la relación directa entre las expresiones paramétricas y el modelado geométrico.

4.1. Objetivos Específicos

- **Sintetizar** el fundamento teórico de las curvas de Bézier de forma estructurada y detallada, mediante un lenguaje accesible para asegurar la comprensión matemática por parte del lector.
- **Calcular** la expresión paramétrica y la regla de correspondencia de la curva en función de las coordenadas exactas de los puntos de control proporcionados.
- **Evaluar** numéricamente la trayectoria matemática para distintos valores del parámetro de tiempo t dentro del intervalo cerrado $[0, 1]$.
- **Representar** gráficamente la trayectoria final de la curva junto con sus respectivos puntos de control, prescindiendo de elementos visuales secundarios para mantener la atención en la forma principal.
- **Diseñar** una interfaz de usuario interactiva e intuitiva con el propósito de facilitar el aprendizaje visual y la comprensión geométrica de las personas que utilicen el programa.

5. Fundamento Teórico

Para comprender la formulación matemática de las curvas de Bézier, resulta esencial repasar dos conceptos matemáticos fundamentales.

Polinomio:

Un polinomio es una expresión algebraica formada por la suma o resta de un número finito de monomios, cada uno compuesto por un coeficiente y una o más variables elevadas a exponentes enteros no negativos.

$$\sum_{i=0}^n a_i x^i = a_0 + a_1 x + a_2 x^2 + \cdots + a_n x^n$$

En donde el grado es el mayor exponente al que esta elevada la variable en cualquiera de sus monomios

Lineal: $f(x) = x + 1$

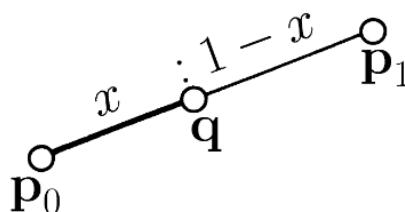
Cuadrática: $g(x) = x^2 + x + 1$

Cúbico: $h(x) = x^3 + x^2 + x + 1$

Cuartico: $i(x) = x^4 + x^3 + x^2 + x + 1$

Interpolación Lineal

La interpolación lineal constituye el bloque de construcción fundamental de este sistema. Dados dos puntos espaciales P_0 y P_1 , una curva lineal de Bézier se define como la línea recta que los conecta. Matemáticamente, este proceso se describe como una suma ponderada de puntos donde cada uno tiene un peso específico determinado por el parámetro t .



De acuerdo con Paluszny, et al (2005), este tipo de operación se denomina formalmente combinación afin. Para que una suma de puntos sea considerada una combinación afin, es requisito indispensable que la suma de sus coeficientes sea exactamente igual a 1.

En la expresión:

$$B(t) = (1 - t)P_0 + tP_1, \quad t \in \mathbb{R}$$

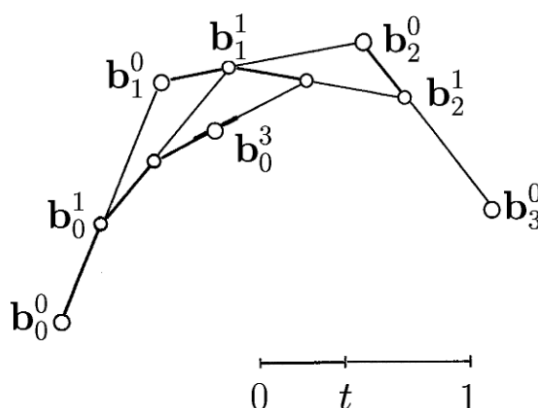
Se observa que los coeficientes $(1-t)$ y t siempre suman la unidad.

Algoritmo de Casteljau

El algoritmo de Casteljau es el método fundamental para evaluar y trazar curvas de Bézier. De acuerdo con Paluszny, (2005) este procedimiento destaca por su simplicidad geométrica y su alta estabilidad numérica. el algoritmo opera a través de interpolaciones lineales repetidas de manera recursiva.

Para comprender el algoritmo de forma sencilla, tomemos como ejemplo una curva cúbica. Esta curva requiere cuatro puntos de control iniciales El objetivo del método es encontrar la ubicación exacta de un punto sobre la curva para un parámetro específico t (un valor numérico entre 0 y 1, donde 0 es el inicio de la curva y 1 es el final).

1. Traza líneas rectas virtuales entre los puntos control originales $b_0^0; b_1^0; b_2^0; b_3^0$ se ubica los puntos intermedios $b_0^1; b_1^1; b_2^1$.
2. Se conecta estos tres nuevos puntos para formar dos nuevas rectas. Al evaluar estas rectas nuevamente con el mismo valor t , se obtienen dos puntos adicionales $b_0^2; b_1^2$.
3. Finalmente, Al aplicar la misma proporción t sobre esta recta final, se obtiene la coordenada exacta del punto del punto b_0^3 ocurre sobre la única recta trazada entre $b_0^2; b_1^2$



Curvas de Bézier

Las curvas de Bézier son un modelo matemático utilizado para generar trazados suaves y continuos mediante el uso de puntos de control. En lugar de definir la curva mediante una función tradicional $y = f(x)$, se definen mediante ecuaciones paramétricas, lo que permite una flexibilidad total en el diseño gráfico y la ingeniería.

1. Definición Analítica

Analíticamente, una curva de Bézier se define como una combinación lineal de puntos de control. Si tenemos un conjunto de $n + 1$ puntos de control $\{P_0, P_1, \dots, P_n\}$, la curva $C(t)$ está determinada por la suma ponderada de estos puntos a lo largo de un parámetro de tiempo t .

$$C(t) = \sum_{i=0}^n P_i \cdot B_{i,n}(t)$$

En el que:

P_i : Son las coordenadas de los puntos de control ingresados por el usuario (vectores en el plano).

t : Es el parámetro de control, definido estrictamente en el intervalo $[0, 1]$.

$B_{i,n}(t)$: Son las funciones base (polinomios de Bernstein) que determinan cuánto aporta cada punto de la curva en un momento dado.

2. Comportamiento del parámetro t:

El parámetro t actúa como el “recorrido” de la curva:

- Al evaluar $C(0)$, el resultado es siempre el punto inicial P_0 .
- Al evaluar $C(1)$, el resultado es siempre el punto final P_n .
- Para cualquier valor intermedio ($0 < t < 1$), la curva se encuentra dentro de la “envolvente convexa” formada por los puntos de control, permitiendo analizar como su posición modifica la forma resultante.

3. Representación Paramétrica (Regla de Correspondencia) Para tu programa, la regla de correspondencia se descompone en dos funciones escalares, una para cada eje:

$$x(t) = \sum_{i=0}^n x_i \cdot B_{i,n}(t)$$

$$y(t) = \sum_{i=0}^n y_i \cdot B_{i,n}(t)$$

Esta separación permite que la computadora evalúe la posición exacta (x, y) para cada incremento de t , facilitando la generación de la representación gráfica que solicita el proyecto.

4. Propiedades Visuales y Análisis

- Interpolación: La curva interpola (pasa exactamente por) los puntos extremos, pero solo se aproxima a los puntos intermedios.
- Influencia Local: Al mover un punto de control, la forma de la curva cambia dinámicamente, lo que permite realizar el análisis solicitado sobre cómo la variación de los puntos modifica la curva.
- Grado de la Curva: El grado del polinomio es siempre igual al número de puntos de control menos uno ($n = \text{puntos} - 1$).

Polinomios de Bernstein:

Los polinomios de Bernstein son las funciones base de las curvas de Bézier. Su función principal es determinar cuánto peso o influencia tiene cada punto de control P_i en la posición de la curva para un valor específico de t .

1. La fórmula general:

Para una curva de grado n (donde $n = \text{número de puntos} - 1$), el polinomio número i se define como:

$$B_{i,n}(t) = \binom{n}{i} t^i (1-t)^{n-i}$$

En el que:

- **El Coeficiente Binomial** $\binom{n}{i}$: Es una constante que escala la función. Se asegura de que la suma de todos los polinomios en cualquier momento t sea exactamente **1** (una propiedad llamada *Partición de la Unidad*). Se desarrolla de la siguiente manera:

$$\binom{n}{i} = \frac{n!}{i!(n-i)!}$$

- **El término de "Crecimiento" (t^i):** A medida que el tiempo t avanza hacia 1, este término aumenta la importancia de los puntos finales.
- **El término de "Decaimiento" $((1-t)^{n-i})$:** A medida que t avanza, este término disminuye, reduciendo la importancia de los puntos iniciales.

Construcción de la Función para una Curva de Grado 2 (3 Puntos):

Para ilustrar cómo la sumatoria se convierte en una regla de correspondencia utilizable por el programa, consideremos el caso de una curva definida por tres puntos de control: P_0 , P_1 y P_2 . Al ser tres puntos, el grado de la curva es $n = 2$.

Utilizando la fórmula general, la función $C(t)$ se expande de la siguiente manera:

$$C(t) = P_0 \cdot B_{0,2}(t) + P_1 \cdot B_{1,2}(t) + P_2 \cdot B_{2,2}(t)$$

Sustituyendo los polinomios de Bernstein de grado 2 que explicamos anteriormente, obtenemos la ecuación paramétrica completa:

$$C(t) = P_0(1-t)^2 + P_1(2t(1-t)) + P_2(t^2)$$

Descomposición en los ejes X e Y:

Como nuestro programa debe dibujar esta curva en un plano cartesiano, la función vectorial $C(t)$ se descompone en dos funciones escalares que el código evalúa de forma independiente para cada valor de $t \in [0,1]$:

- Para el eje X: $x(t) = x_0(1-t)^2 + x_1(2t(1-t)) + x_2(t^2)$
- Para el eje Y: $y(t) = y_0(1-t)^2 + y_1(2t(1-t)) + y_2(t^2)$

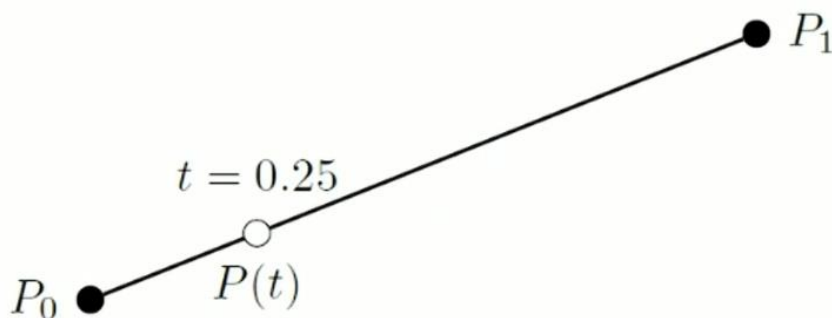
Donde (x_0, y_0) , (x_1, y_1) y (x_2, y_2) son las coordenadas de los puntos de control. Esta es la regla de correspondencia que el programa calcula y muestra al usuario, permitiendo observar cómo cada punto "tira" de la curva con una intensidad distinta según momento t .

Construcción de curvas de Bézier

Curvas lineales de Bézier:

Es una curva recta entre los puntos P_0 y P_1 se representa por la función

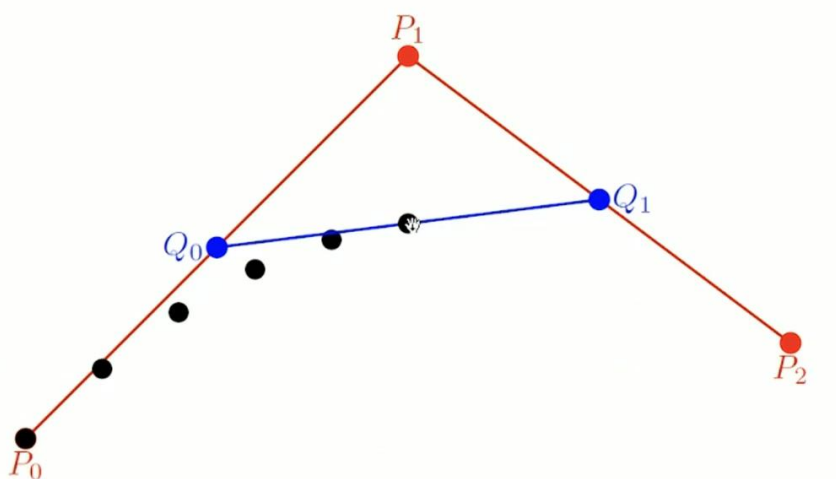
$$B(t) = P_0 + (P_1 - P_0)t = (1-t)P_0 + tP_1, t \in [0; 1]$$



Curvas cuadráticas de Bézier:

Es la curva trazada por la función $B(t)$ con puntos de control P_0, P_1 y P_2 se representa

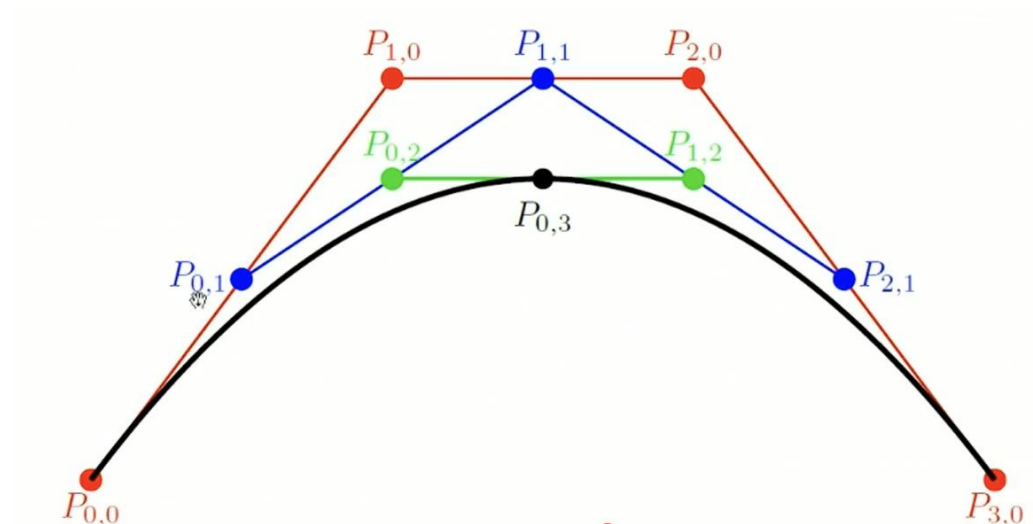
$$B(t) = (1 - t)^2 P_0 + 2t(1 - t)P_1 + t^2 P_2, t \in [0; 1]$$



Curvas cúbicas de Bézier:

Es la curva con cuatro puntos P_0, P_1, P_2 y P_3 permite crear formas más complejas con cambios de dirección gracias a la influencia de dos puntos intermedios P_1 y P_2 . se representa paramétrica mente

$$B(t) = (1 - t)^3 P_0 + 3t(1 - t)^2 P_1 + 3t^2(1 - t)P_2 + t^3 P_3, t \in [0; 1]$$



Curvas de grado n

Al ser generalizada la curva para n se le llama spline:

$$B(t) = \sum_{i=0}^n \binom{n}{i} P_i (1-t)^{n-i} t^i, \quad t \in [0; 1]$$

Representación Matricial de las Curvas de Bézier

Más allá de la suma algebraica de los polinomios de Bernstein, la implementación computacional y el análisis lineal de estas curvas se simplifican mediante el uso de la notación matricial. Esta representación permite expresar la curva de Bézier como el producto de tres matrices fundamentales, lo que facilita el cálculo de coordenadas en entornos de programación de alto rendimiento. Para una curva de grado n , la posición de cualquier punto $C(t)$ se define mediante el siguiente producto:

$$C(t) = T \cdot M \cdot P$$

Donde los componentes se definen de la siguiente manera:

1. **Vector de Parámetros (T):** Es una matriz fila que contiene las potencias descendentes del parámetro t , desde t^n hasta t^0 . Por ejemplo, para una curva cúbica ($n = 3$), el vector es $[t^3, t^2, t, 1]$.
2. **Matriz de Base de Bézier (M):** Es una matriz cuadrada de orden $(n + 1) \times (n + 1)$. Esta matriz es constante para un grado determinado y contiene los coeficientes binomiales distribuidos de forma que, al multiplicarse por el vector T , resultan en los polinomios de Bernstein correspondientes.
3. **Matriz de Puntos de Control (P):** Es una matriz que contiene las coordenadas (x, y) de los puntos de control ingresados por el usuario.

Ejemplo de representación matricial (Curva de Bézier cuadrática $n = 2$):

Cuando el usuario ingresa 3 puntos de control (P_0, P_1, P_2), el programa trabaja con una curva de grado $n = 2$. Para este caso, la estructura $C(t) = T \cdot M \cdot P$ se desarrolla de la siguiente manera:

Desarrollo de los polinomios:

Primero, expandimos los polinomios de Bernstein de grado 2 para identificar los coeficientes de sus potencias:

$$\begin{aligned} B_{0,2}(t) &= (1-t)^2 = 1t^2 - 2t + 1 \\ B_{1,2}(t) &= 2t(1-t) = -2t^2 + 2t + 0 \\ B_{2,2}(t) &= t^2 = 1t^2 + 0t + 0 \end{aligned}$$

Construcción de la Matriz $M_{3 \times 3}$:

Al organizar estos coeficientes en una matriz donde cada columna corresponde a un polinomio y cada fila a una potencia de $t(t^2, t, 1)$, obtenemos la Matriz de Base de Bézier cuadrática:

$$\mathbf{M} = \begin{pmatrix} 1 & -2 & 1 \\ -2 & 2 & 0 \\ 1 & 0 & 0 \end{pmatrix}$$

Ecuación final de la curva con representación matricial:

La posición de la curva en el plano para cualquier valor de t se calcula entonces como:

$$C(t) = \begin{pmatrix} t^2 & t & 1 \end{pmatrix} \cdot \begin{pmatrix} 1 & -2 & 1 \\ -2 & 2 & 0 \\ 1 & 0 & 0 \end{pmatrix} \cdot \begin{pmatrix} x_0 & y_0 \\ x_1 & y_1 \\ x_2 & y_2 \end{pmatrix}$$

5.1. Ejemplo aplicativo 1

Diseño y Renderizado de Tipografía Digital

Antiguamente, las letras eran mapas de bits. Si ampliabas una letra, se veía "escalonada" o pixelada. Además, almacenar cada tamaño de letra ocupaba demasiada memoria.

Las fuentes modernas (Opentype) definen el contorno de cada carácter mediante una serie de curvas de Bézier unidas. Una letra "S", por ejemplo, se construye uniendo varias curvas cúbicas. El sistema solo guarda las coordenadas de los puntos de control.

Esquemas de TrueType y PostScript. Los contornos de una fuente se describen mediante una serie de puntos. Para describir una línea recta, basta con especificar las coordenadas de los dos puntos extremos; sin embargo, para describir una curva, se necesitan puntos intermedios. Para ello, se utiliza una ecuación matemática denominada curva de Bézier (Murray, 2007)

Esto permite la Independencia de Resolución. Cuando el software necesita mostrar una letra, el procesador calcula la curva matemáticamente en tiempo real para adaptarla a los píxeles exactos de la pantalla o la impresora, garantizando bordes perfectamente suaves a cualquier escala.

Ejemplo: Trazado de un "Logotipo Curvo"

Imagina que quieres diseñar la parte inferior de una "U" perfectamente suave usando una Curva de Bézier Cuadrática.

1. Definición de Parámetros:

- Punto inicial $P_0: (-2, 2)$
- Punto de control $P_1: (0, -2)$
- Punto final $P_2: (2, 2)$

2. Resolución mediante la fórmula:

La fórmula para una curva cuadrática es: $B(t) = (1 - t)^2 P_0 + 2t(1 - t)P_1 + t^2 P_2$ donde t va de 0 a 1.

- Paso A: Inicio, $t = 0$, al sustituir el resultado es $(-2, 2)$. La curva inciaa excatamente en P_0
- Paso B: Punto medio, $t = 0.5$ calculamos la posición en el centro del trazo
- $x = (0.5)^2(-2) + 2(0.5)(0.5)(0) + (0.5)^2(2) = -0.5 + 0 + 0.5 = 0$
- $y = (0.5)^2(2) + 2(0.5)(0.5)(-2) + (0.5)^2(2) = 0.5 - 1 + 0.5 = 0$
- Resultado, el punto más bajo de la curva está en $(0, 0)$. Nota que no toca el punto de control $(0, -2)$, pero se orienta hacia él.
- Paso C: Punto final $t = 1$, el resultado es $(2, 2)$

3. Visualización práctica:

Al unir todos los puntos obtenidos variando t (0.1, 0.2, 0.3, ...), obtienes una curva simétrica y suave que un diseñador puede alterar simplemente moviendo P_1

5.2. Ejemplo aplicativo 2

Suavizado para trayectoria de un robot

En la agricultura de precisión, se utilizan robots para identificar y eliminar maleza de forma automatizada. El robot Ceres AgroBot emplea un algoritmo de búsqueda llamado A* para encontrar el camino más corto entre el punto inicial de la herramienta y la maleza a eliminar.

Aunque el algoritmo A* encuentra una ruta eficiente, esta suele estar compuesta por segmentos de línea recta con ángulos muy cerrados. Si un robot intentara seguir esa trayectoria directamente, los actuadores (motores) sufrirían desplazamientos y acciones bruscas, lo que afectaría la precisión y la vida útil del sistema mecánico.

Dentro de la planificación de movimientos robóticos, los algoritmos de búsqueda como A* suelen generar rutas con cambios de dirección repentinos. Para mitigar este problema, Solaque et al. (2018) proponen un proceso de 'alisamiento' mediante curvas de Bézier, asegurando que la herramienta del robot Ceres AgroBot se desplace de forma fluida hacia la maleza sin colisionar con el cultivo. Este enfoque demuestra que las curvas de Bézier no son solo herramientas estéticas, sino soluciones críticas para la eficiencia dinámica de los actuadores en sistemas de agricultura de precisión.

A continuación, se explica como se ha implementado en el trabajo de Planificación de trayectorias por técnica de A* y suavizado por curvas de Bézier para la herramienta del sistema de remoción de maleza de un robot dedicado a labores de agricultura de precisión

- Discretización: El espacio de trabajo se divide en una nube de puntos 3D.
- Identificación de Obstáculos: Las plantas de interés se encapsulan en "cubos" que el robot no debe tocar.
- Cálculo de Puntos de Control: Los nodos o puntos clave entregados por el algoritmo A* se toman como puntos de anclaje.
- Parametrización: Se aplica la fórmula de Bézier para unir esos puntos, transformando los ángulos rectos en curvas suaves que mantienen la continuidad del movimiento en el tiempo.

6. Código del programa (Avance del 30%)

```

import matplotlib.pyplot as plt from matplotlib.animation import FuncAnimation import
numpy as np import math
--- FUNCIONES MATEMÁTICAS ---
def nCr(n, r): return math.comb(n, r)
def polinomio_bernstein(i, n, t):
return nCr(n, i) * (t ** i) * (1 - t) ** (n - i)

def calcular_con_pesos(puntos_control, t_valores):
n = len(puntos_control) - 1
curva = []
pesos_totales = []

for t in t_valores:
punto_t = np.array([0.0, 0.0])
pesos_t = []
for i in range(len(puntos_control)):
peso = polinomio_bernstein(i, n, t)
pesos_t.append(peso)
punto_t += peso * np.array(puntos_control[i])
curva.append(punto_t)
pesos_totales.append(pesos_t)

return np.array(curva), np.array(pesos_totales)

try: n_puntos = int(input("\nDefine la cantidad de puntos de control: ")) puntos = [] for i in
range(n_puntos): x = float(input(f" Coordenada X del punto P{i}: ")) y = float(input(f"
Coordenada Y del punto P{i}: ")) puntos.append([x, y])
resolucion = int(input("¿Resolución de la curva? (pasos de t): "))

```

```
except ValueError: print("Error: Ingresar números válidos.") exit()
```

Pre-cálculo de datos

```
t_valores = np.linspace(0, 1, resolucion) curva, pesos = calcular_con_pesos(puntos, t_valores)
```

--- CONFIGURACIÓN DE LA ANIMACIÓN ---

```
fig, ax = plt.subplots(figsize=(10, 7)) ax.grid(True, linestyle=':', alpha=0.6)
```

```
ax.set_title(f'Influencia de Vectores de Control (Bézier Grado {n_puntos - 1})')
```

Elementos estáticos (el fondo)

```
px, py = zip(*puntos) ax.plot(px, py, '--', color='gray', alpha=0.3, label='Polígono de Control')
```

```
for i, p in enumerate(puntos): ax.text(p[0], p[1] + 0.3, f'P {i}', fontweight='bold')
```

Elementos que se actualizarán (la animación)

```
linea_curva, = ax.plot([], [], color='red', linewidth=3, label='Curva de Bézier') punto_actual, =  
ax.plot([], [], 'ro', markersize=8)
```

Creamos una lista para guardar las "líneas de influencia" (rubber bands)

Usaremos colores distintos para identificar qué vector está 'jalando'

```
colores_influencia = plt.cm.tab10(np.linspace(0, 1, n_puntos)) lineas_influencia = [] for i in  
range(n_puntos): ln, = ax.plot([], [], color=colores_influencia[i], linestyle='-', linewidth=1,  
alpha=0) lineas_influencia.append(ln)
```

```
ax.legend(loc='upper left')
```

Función de inicialización

```
def init(): linea_curva.set_data([], []) punto_actual.set_data([], []) for ln in lineas_influencia:  
ln.set_data([], []) ln.set_alpha(0) # Invisible al inicio return [linea_curva, punto_actual] +  
lineas_influencia
```

Función que actualiza cada frame (cuadro)

```
def update(frame): # Punto actual sobre la curva B(t) bx, by = curva[frame, 0], curva[frame, 1]  
punto_actual.set_data([bx], [by])
```

```
# Dibujar el rastro de la curva hasta este punto  
linea_curva.set_data(curva[:frame + 1, 0], curva[:frame + 1, 1])
```

```
# Visualizar el 'proceso de combinación lineal' de Bernstein  
pesos_frame = pesos[frame]  
n = len(puntos) - 1
```

```

# Actualizar las rubber bands (líneas de influencia)
for i in range(len(puntos)):
    px_vector, py_vector = puntos[i][0], puntos[i][1]

    # Línea del punto en la curva B(t) hacia el vector P_i
    lineas_influencia[i].set_data([bx, px_vector], [by, py_vector])

    # El peso B_{i,n}(t) controla la opacidad (alpha) de la línea
    # Si el peso es alto, la línea es muy visible. Si es bajo, desaparece.
    lineas_influencia[i].set_alpha(pesos_frame[i])

return [linea_curva, punto_actual] + lineas_influencia

```

Crear la animación

```

anim = FuncAnimation(fig, update, frames=resolucion, init_func=init, blit=True, interval=50)
print("\n[INFO] Animación generada. Cierra la ventana para continuar.") plt.show()

```

7. Pantallazos de la ejecución del programa y su explicación

7.1. Imagen 1 y su explicación

```
import matplotlib.pyplot as plt
from matplotlib.animation import FuncAnimation
import numpy as np
import math
```

- Librerías utilizadas para conseguir hallar la curva que se solicita. Usamos math y numpy para los calculos necesarios en el polinomio de Berstein y la suma de los puntos. Se uso matplotlib para el dibujo de la curva, tanto pyplot como animation, de forma que se pueda graficar el componente estático y animado.

7.2. Imagen 2 y su explicación

```
def nCr(n, r): 1 usage
    return math.comb(n, r)

def polinomio_bernstein(i, n, t): 1 usage

    return nCr(n, i) * (t ** i) * (1 - t) ** (n - i)
```

- Se utiliza la función “nCr(n, r)” para definir el coeficiente binomial necesario para calcular peso de cada punto en un tiempo determinado para hallar la ubicación del de la curva en ese tiempo.
- La función “polinomio_bernstein(i, n, t)” sirve para calcular precisamente el valor de cada peso. Donde t representa el dominio basado en el tiempo, n representa el grado polinomial y la i representa la posición de cada punto.

7.3. Imagen 3 y su explicación

```
def calcular_con_pesos(puntos_control, t_valores): 1 usage

    n = len(puntos_control) - 1
    curva = []
    pesos_totales = []

    for t in t_valores:
        punto_t = np.array([0.0, 0.0])
        pesos_t = []
        for i in range(len(puntos_control)):
            peso = polinomio_bernstein(i, n, t)
            pesos_t.append(peso)
            punto_t += peso * np.array(puntos_control[i])
        curva.append(punto_t)
        pesos_totales.append(pesos_t)

    return np.array(curva), np.array(pesos_totales)
```

- La función “calcular_con_pesos...” es de vital importancia, dado que es la principal operación donde las coordenadas se convierten en vectores que operan entre sí.
 - Primero ayuda a definir el grado del polinomio restando 1 a la cantidad de puntos seleccionados.
 - Se crean arreglos vacíos para contener los valores tanto de la curva como de los pesos totales.
 - El bucle “for t in valores” permite evaluar cada punto en cada determinado tiempo que el usuario ingrese. De esta forma t_valores permitira ver cuántos puntos ayudarán a conformar la curva deseada. Después “punto_t” se encargará de almacenar las coordenadas, como puede verse en la imagen empieza en (0,0). También se incluye un arreglo vacío para contener cada uno de los pesos.
 - A continuación, se incluye un nuevo bucle que se encargue de las operaciones necesarias para conseguir los arreglos que contengan los valores para la construcción de la curva. Por ejemplo “peso” será igual al valor que se obtenga de resolver el polinomio de Bernstein, este será añadido al arreglo de pesos_t y luego se sumará el punto_t para pasar al siguiente punto. También se añaden valores a la curva y a pesos_totales.
 - Finalmente, la función devuelve tanto el array curva como el array de pesos_totales.

7.4. Imagen 4 y su explicación

```
try:
    n_puntos = int(input("\nDefine la cantidad de puntos de control: "))
    puntos = []
    for i in range(n_puntos):
        x = float(input(f" Coordenada X del punto P{i}: "))
        y = float(input(f" Coordenada Y del punto P{i}: "))
        puntos.append([x, y])

    resolucion = int(input("¿Resolución de la curva? (pasos de t): "))
except ValueError:
    print("Error: Ingresar números válidos.")
    exit()
```

- Esta parte del código permite el ingreso de los valores necesarios para la graficación de la curva. Se solicita al usuario el número de puntos así como las coordenadas de cada uno. Se incluye un criterio de excepción para corroborar que los datos ingresados sean números válidos.

7.5. Imagen 5 y su explicación

```
# Pre-cálculo de datos
t_valores = np.linspace( start: 0, stop: 1, resolucion)
curva, pesos = calcular_con_pesos(puntos, t_valores)

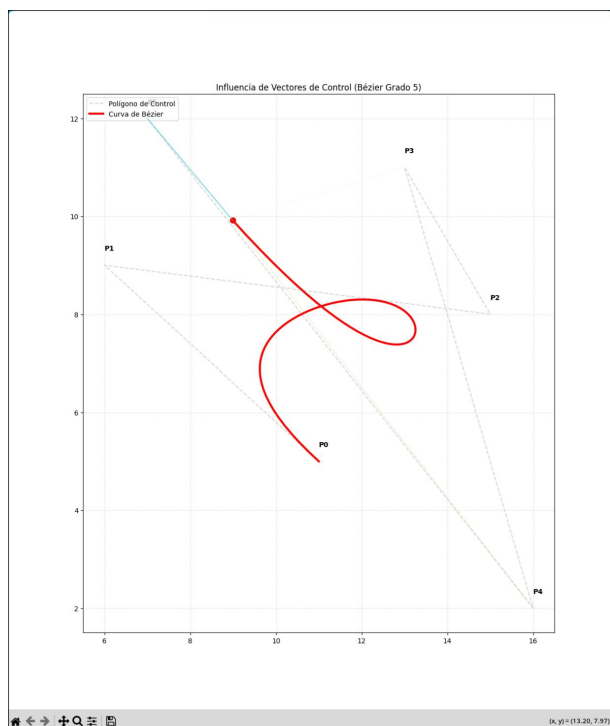
# --- CONFIGURACIÓN DE LA ANIMACIÓN ---
fig, ax = plt.subplots(figsize=(10, 7))
ax.grid( visible: True, linestyle=':', alpha=0.6)
ax.set_title(f"Influencia de Vectores de Control (Bézier Grado {n_puntos - 1})")

# Elementos estáticos (el fondo)
px, py = zip(*puntos)
ax.plot( *args: px, py, '--', color='gray', alpha=0.3, label='Polígono de Control')
for i, p in enumerate(puntos):
    ax.text(p[0], p[1] + 0.3, s: f'P{i}', fontweight='bold')

# Elementos que se actualizarán (la animación)
linea_curva, = ax.plot( *args: [], [], color='red', linewidth=3, label='Curva de Bézier')
punto_actual, = ax.plot( *args: [], [], 'ro', markersize=8)
```

- Aquí se calculan los valores iniciales para el gráfico que se hará de la curva. Aquí aparecen en gran mayoría elementos de matplotlib. Se definen el color el label que acompañara al gráfico principal.

7.6. Imagen 6 y su explicación



- Gráfico resultante después de la compilación del código antes descrito.

8. Bibliografía

Galdames, O. (2011). Modelización con curvas y superficies de Bézier. *Revistas Científicas UPV*, 4(14), 181-193.

<https://polipapers.upv.es/index.php/MSEL/article/view/3071>

Profesores UPC. (2022). *Curvas de Bézier*. Álgebra Lineal - Libro digital UPC.

Prautzsch, H., Boehm, W., & Paluszny, M. (2005). *Métodos de Bezier y B-Splines*. KIT Scientific Publishing.

Farin, G. (1997). *Curves and Surfaces for Computer-Aided Geometric Design: A Practical Guide* (4.ª ed.). Academic Press.

http://lib.ysu.am/disciplines_bk/37ba5d3ec9d802d344329876c50be2b0.pdf

regCode. (2021). *Curvas de Bézier (Bézier/Casteljau)*. Youtube.

<https://www.youtube.com/watch?v=dF1rR2Px8CA>

Solaque, L., Velasco, A., & Riveros, A. (2018). Planificación de trayectorias por técnica de A* y suavizado por curvas de Bezier para la herramienta del sistema de remoción de maleza de un robot dedicado a labores de agricultura de precisión. *Entre Ciencia E Ingeniería*, 12(24), 43-51. <https://doi.org/10.31908/19098367.3814>

Murray, A. (2007, 8 de septiembre). *PostScript, TrueType and OpenType: Curves and Outlines*. CR8 Software Solutions. <https://cr8software.net/article004.html>

9. Anexos

Enlace al código completo y reuniones:
<https://drive.google.com/drive/folders/1QR2FGtWgYFyCQNww16emQahrCgWseAIN?usp=sharing>